



Xarxes neuronals i *deep learning*

Lluís A. Belanche

Computer Science Department
Universitat Politècnica de Catalunya
belanche@cs.upc.edu

LECTURE 3: Feed-forward neural networks (I)

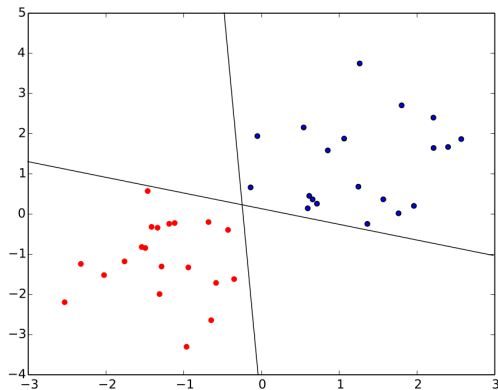
Version of March 3, 2025

Feed-forward neural networks (I)

- 1 Linear networks (I): the Perceptron and its optimization. Variants. Convergence theorem.
- 2 Lippmann interpretation of Perceptron networks.
- 3 Linear networks (II): the Delta rule and its optimization. Variants.
- 4 Linear networks $\leftrightarrow$ GLMs. Activation functions and error (or loss) functions. Summary table.

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron



Two classes, no distributional or *a priori* assumptions, linear separability (“linsep”)

The goal is to **generalize** well to unseen data (as well as possible, topped by the **Bayes error**)

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron

We have a data set $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, with $\mathbf{x}_i \in \mathbb{R}^m$ and $y_i \in \{-1, +1\}$, describing a two-class problem.

We wish to find a linear function f which best models D :

- Set up an **affine function** $g(\mathbf{x}) := \omega^\top \mathbf{x} + b$
- Obtain a **linear discriminant** as $f(\mathbf{x}) := \text{sgn}(g(\mathbf{x}))$
- We would like to find an iterative learning procedure for ω, b such that:
 $\omega^\top \mathbf{x}_i + b > 0$, when $y_i = +1$
 $\omega^\top \mathbf{x}_i + b < 0$, when $y_i = -1$
that is $y_i(\omega^\top \mathbf{x}_i + b) > 0$ or simply
 $y_i g(\mathbf{x}_i) > 0 \quad (1 \leq i \leq n)$

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron

Definition

The functional margin γ_i^F of an example $(\mathbf{x}_i, y_i)$ wrt an hyperplane ω, b is

$$\gamma_i^F := y_i g(\mathbf{x}_i) = y_i(\omega^\top \mathbf{x}_i + b)$$

We note that $\gamma_i^F > 0$ iff $(\mathbf{x}_i, y_i)$ is correctly classified by the hyperplane.

Definition

The geometric margin γ_i^G of an example $(\mathbf{x}_i, y_i)$ is the functional margin wrt the hyperplane $\frac{\omega}{\|\omega\|}, \frac{b}{\|\omega\|}$.

We note that $\gamma_i^G = d(\mathbf{x}_i, \pi)$, where π stands for the hyperplane $\pi : \omega^\top \mathbf{x} + b = 0$.

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron

Definition

The margin Γ of a dataset D is the maximum geometric margin over all possible hyperplanes:

$$\Gamma(D) := \sup_{(\omega, b) \in \mathbb{R}^m \times \mathbb{R}} \min_{i=1, \dots, n} \gamma_i^G(\omega, b)$$

Any hyperplane such that:

- 1 it realises Γ on D
- 2 all its functional margins $\gamma_i^F > 0$

is known as a **maximum margin hyperplane**.

We set for the goal of learning a hyperplane satisfying 2. What algorithm satisfies both goals? the hard-margin SVM

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron

Definition

The margin Γ of a dataset D is the maximum geometric margin over all possible hyperplanes:

$$\Gamma(D) := \sup_{(\omega, b) \in \mathbb{R}^m \times \mathbb{R}} \min_{i=1, \dots, n} \gamma_i^G(\omega, b)$$

Any hyperplane such that:

- 1 it realises Γ on D
- 2 all its functional margins $\gamma_i^F > 0$

is known as a **maximum margin hyperplane**.

We set for the goal of learning a hyperplane satisfying 2. What algorithm satisfies both goals? the hard-margin SVM

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron

Definition

The Perceptron cost function is defined as:

$$J(\omega, b) := - \sum_{i/\mathbf{x}_i \in M} y_i(\omega^\top \mathbf{x}_i + b) = - \sum_{i/\mathbf{x}_i \in M} \gamma_i^F$$

being M the set of missclassified examples.

Proposition

- 1 $J \geq 0$; moreover $M = \emptyset \Rightarrow J = 0$
- 2 J is continuous and piece-wise linear

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron

Extending inputs and weights to incorporate a 1 (inputs) and b (weights) so that the algorithm works in one extra dimension, a gradient descent updating rule leads to Rosenblatt's famous **Perceptron learning algorithm**.

```
initialize:  set  $\omega(0)$ 
              $t := 0$ 
             repeat
                 For  $i = 1, \dots, n$ 
                     Predict  $\hat{y}_i(t) := \text{sgn}(\omega^\top(t)\mathbf{x}_i)$ 
correct mistake:  if  $y_i\hat{y}_i(t) \leq 0$  then
                      $\omega(t+1) := \omega(t) + y_i\mathbf{x}_i$ 
                      $t := t + 1$ 
                 end
             until no mistakes made within the for loop
result:   $\omega(t)$ 
```

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron

Theorem

(Novikoff'62; Block'62) Given a data set describing a linearly separable two-class problem $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, with $\mathbf{x}_i \in \mathbb{R}^m$ and $y_i \in \{-1, +1\}$, s.t. $\|\mathbf{x}_i\| \leq R$, for all $1 \leq i \leq n$.

Suppose there exists a vector ω^* fulfilling $\|\omega^*\| = 1$ and, for some γ , $y_i(\omega^{*\top} \mathbf{x}_i) \geq \gamma > 0$ holds.

Then the number of mistakes t made by the Perceptron learning algorithm (until such a solution is found) is at most

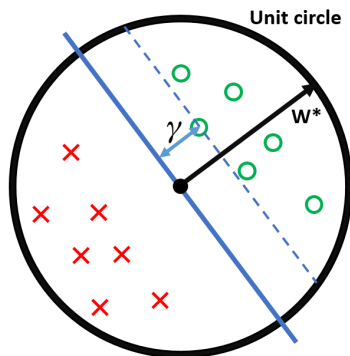
$$\frac{R^2}{\gamma^2}$$

F. Rosenblatt, "The Perceptron—a perceiving and recognizing automaton", Cornell Aeronautical Laboratory, Report 85-460-1, 1957.

A. B. Novikoff, "On convergence proofs on perceptrons", Symposium on the Mathematical Theory of Automata, 12, 615-622, 1962.

NEURAL NETWORKS AND DEEP LEARNING

The Perceptron



Example for the case of normalized input data $\|\mathbf{x}\|^2 = 1$, so $R = 1$.